CS791 Assignment3 [Team: MSD]
Dr. Shashi Tharoor and LLMs

Madhav Kotecha (24M0833)*
Smita Gautam (24M0845)*
Deeksha Koul (24D0365)*
*Equal contribution

October 22nd, 2025

1 Task 0: Primer on LLM Decoding Techniques

We tested three decoding strategies here: **Greedy**, **Temperature Sampling** ($T = 1.0, 0.9, 0.5$), and **Top-k Sampling** ($k = 5, 10$). Just setting a baseline before the main tasks.

Greedy – picks the highest probability token every time. Deterministic, single output. Temperature sampling – adds randomness by tweaking how peaked/flat the distribution gets via T. Top-k – restricts choices to top k tokens only at each step.

We tracked three metrics: *Expected Reward*, *Perplexity*, *Entropy*. Results in Table 1.

- **Greedy**: Takes max probability token always. Lowest perplexity and reward both – fluent yes, but zero diversity or exploration.
- **Random Sampling ($T = 1.0$)**: Neutral temperature. Very high variance in outputs (entropy peaks), best reward overall. Trade-off? Perplexity shoots up – less fluent text.
- **Random Sampling ($T = 0.5$)**: Lower T = less random. Distribution gets sharper. Entropy and perplexity both drop vs $T = 1.0$. Falls somewhere between greedy and full randomness.
- **Random Sampling ($T = 0.9$)**: Middle option – some exploration without losing too much structure. Metrics between $T = 0.5$ and $T = 1.0$. Reward and entropy increase as T goes up, as expected.
- **Top-k Sampling ($k = 5$)**: Only top 5 tokens considered – reduces wild randomness. Slight reward and entropy bump over greedy. Perplexity stays reasonable.

- **Top-k Sampling ($k = 10$)**: More tokens = more exploration space. Reward and entropy tick up a bit, perplexity increases slightly too.

Overall Analysis: Trade-offs everywhere – reward vs fluency vs diversity. Greedy = most fluent, least diverse. $T = 1.0$ = max diversity and reward, but fluency suffers. Middle ground ($T = 0.5$, $T = 0.9$, Top-k) works well – controlled randomness, better reward, doesn't tank fluency too much.

Task 0	**Expected Reward**	**Perplexity**	**Entropy**
Greedy	5.8391	3.3650	5.1068
Random sampling (temp = 1.0)	8.8926	10.3835	6.0937
Random sampling (temp = 0.5)	6.4342	4.1027	5.6661
Random sampling (temp = 0.9)	7.4854	6.6723	5.9010
Top-k sampling ($k = 5$)	6.5820	5.3060	5.7741
Top-k sampling ($k = 10$)	7.0706	6.2797	5.8508

Table 1: Performance comparison on Task 0 using different decoding strategies.

2 Task 1: Sequential Importance Sampling (SIS)

Task 1 builds on baseline – now using **Sequential Importance Sampling (SIS)**. Generate continuations via top-k proposal, then reweight by reward-scaled importance weights $w(x) \propto \exp(\beta R(x))$. β controls how much reward matters.

Tested $k \in \{5, 10\}$ and $\beta \in \{0.5, 1.0, 5.0, 10.0\}$.

Figure 1 shows normalized importance weight histograms for all (k, β) pairs. Weights are heavily skewed in every setting – just a few samples get most of the mass. Gets worse as β increases. High reward scaling basically amplifies top sequences, kills effective diversity.

Table 2 has the numbers. Both $k = 5$ and $k = 10$: **Expected Reward** goes up consistently with higher β. So yeah, SIS does push toward better rewards.

For $k = 5$: **Perplexity drops** – 5.41 at $\beta = 0.5$ down to 5.27 at $\beta = 10$. More reward emphasis = sampled text becomes more probable under base model too.

For $k = 10$: opposite trend, perplexity increases (6.66 to 7.51). Wider proposal space brings in more exploratory but less likely sequences.

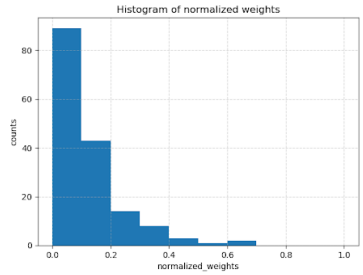
Entropy basically flat within each k. Diversity mostly comes from the proposal distribution, not the reweighting.

So – higher β = better reward focus, but weights get super concentrated at extreme values. Diminishing returns kick in. Larger k = slightly better reward and entropy. Trade-off between exploration and control, nothing dramatic.

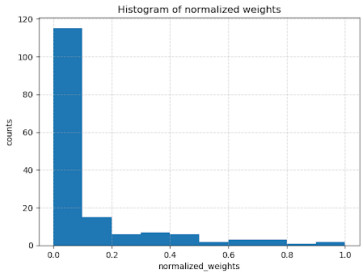
Bottom line: SIS extends baseline decoding nicely – prioritizes high-reward stuff without completely trashing fluency. But crank β too high and effective diversity tanks.

Task 1	Expected Reward	Perplexity	Entropy
$k = 5$, $\beta = 0.5$	8.0516	5.41	5.7742
$k = 5$, $\beta = 1.0$	8.8458	5.36	5.7742
$k = 5$, $\beta = 5$	9.4254	5.28	5.7742
$k = 5$, $\beta = 10$	9.4418	5.27	5.7742
$k = 10$, $\beta = 0.5$	8.2985	6.66	5.8509
$k = 10$, $\beta = 1.0$	8.9772	6.95	5.8509
$k = 10$, $\beta = 5$	9.5350	7.38	5.8509
$k = 10$, $\beta = 10$	9.5590	7.51	5.8509

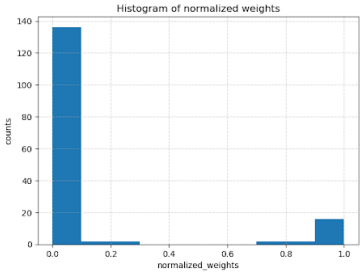
Table 2: Performance comparison on Task 1 with varying k and β values.



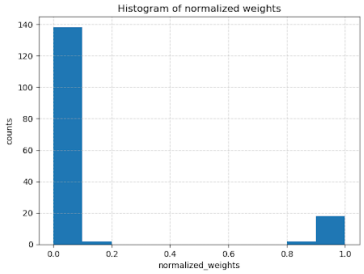
(a) $k = 5$, $\beta = 0.5$



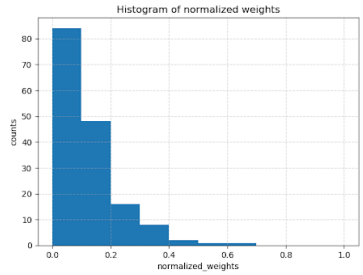
(b) $k = 5$, $\beta = 1.0$



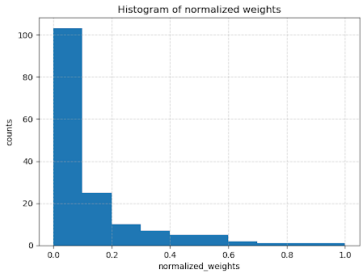
(c) $k = 5$, $\beta = 5$



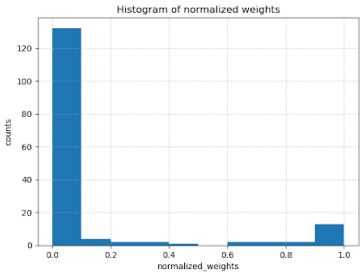
(d) $k = 5$, $\beta = 10$



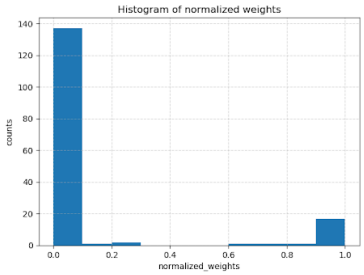
(e) $k = 10$, $\beta = 0.5$



(f) $k = 10$, $\beta = 1.0$



(g) $k = 10$, $\beta = 5$



(h) $k = 10$, $\beta = 10$

3 Task 2: Sequential Monte Carlo Sampling (SMC)

Implementation logic for weights:
We are given $\pi_t(x_{1:t}) = P_{\text{llama}}(x_{1:t})\, e^{\beta R(x_{1:t})}$
The proposal distribution q_t is top-k sampling from P_{llama} and can be described as: $q_t(x_t \mid x_{1:t-1}) = \text{Top-}k$ normalized $P_{\text{llama}}(x_t \mid x_{1:t-1})$

The incremental importance weight becomes:

$$w_t = \frac{\pi_t(x_{1:t})}{\pi_{t-1}(x_{1:t-1})\, q_t(x_t \mid x_{1:t-1})}$$

Substitute $\pi_t(x_{1:t}) = P_{\text{llama}}(x_{1:t})e^{\beta R(x_{1:t})}$:

$$w_t = \frac{P_{\text{llama}}(x_{1:t})e^{\beta R(x_{1:t})}}{P_{\text{llama}}(x_{1:t-1})e^{\beta R(x_{1:t-1})}\, q_t(x_t \mid x_{1:t-1})}$$

we know: $\frac{P_{\text{llama}}(x_{1:t})}{P_{\text{llama}}(x_{1:t-1})} = P_{\text{llama}}(x_t \mid x_{1:t-1})$
So:

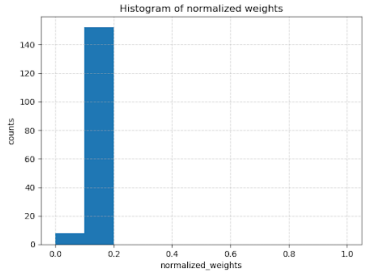
$$w_t = e^{\beta(R_t - R_{t-1})} \frac{P_{\text{llama}}(x_t \mid x_{1:t-1})}{q_t(x_t \mid x_{1:t-1})}$$

Resampling: at each time step $t < T$, we use multinomial sampling based on the normalized weights $\tilde{w}_t^{(i)}$. We pick N particles according to these weights, copy their sequences, rewards, and completion flags('is_particle_done' tracks this), and then add the newly sampled token from the chosen index to each sequence.
After this, the new particle set replaces the old one, and all weights are reset to uniform $1/N$.

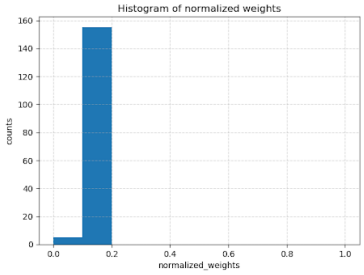
Observations from Table 3 [Task 2.]: In this task, using multiple particles we can observe that the average reward is higher compared to the earlier setup of top-k sampling(k=10). As β increased, more emphasis is done on reward function, hence the particles move towards high -reward paths, which is expected from the setup.
Perplexity and entropy also went up a bit, although entropy does not show a big change, implying that even in high-reward situations(of Task2.) , we still have diversity(due to resampling step).
Overall, SMC managed a good mix of exploration of different continuations and also exploiting the ones with higher rewards.
Histograms Task 2: Here we can see that there is some weight collapse early on in ($\beta = 0.5, 1.0$). Although for a higher β, higher reward, the weight distribution becomes more uneven but still focus seems to be just on high-reward samples.

Task 2	Expected Reward	Perplexity	Entropy
$\beta = 0.5$	8.7448	6.37	5.5414
$\beta = 1.0$	9.1566	6.39	5.5258
$\beta = 5$	14.4933	7.53	5.6839
$\beta = 10$	15.5024	9.67	5.7643

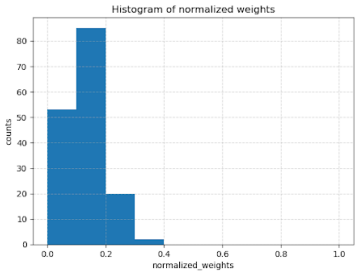
Table 3: Performance comparison on Task 2 for different β values.



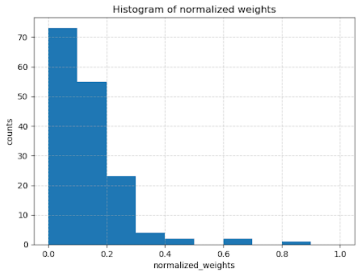
(a) $\beta = 0.5$



(b) $\beta = 1.0$



(c) $\beta = 5$



(d) $\beta = 10$

Figure 2: Histograms of Task 2 results for different β values.

4 Task 3: Twisted Sequential Monte Carlo Sampling (TSMC)

This task extends Task 2. by introducing a *twist function* ϕ_t that looks one step ahead and adjusts particle weights based on the expected future reward. The overall target distribution:

$$\pi_t(x_{1:t}) \propto P_{\text{llama}}(x_{1:t})\, e^{\beta R(x_{1:t})}\, \phi_t(x_{1:t})$$

and the incremental weight update includes the ratio of successive twist terms:

$$w_t = e^{\beta(R_t - R_{t-1})} \frac{P_{\text{llama}}(x_t \mid x_{1:t-1})}{q_t(x_t \mid x_{1:t-1})} \frac{\phi_t(x_{1:t})}{\phi_{t-1}(x_{1:t-1})}.$$

Furthermore, the twist function ϕ_t is estimated using a bigram model. Although we are provided with only trigram probability distribution, we implement bigram from the available trigram counts before running TSMC.

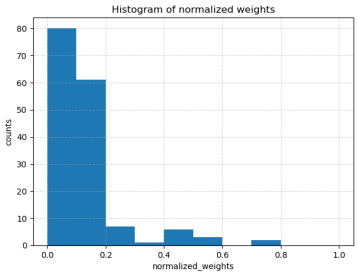
Apart from this, the sampling procedure is mostly the same as in Task 2: particles are sampled using top-k from the model, weights are updated with the additional twist ratio term, and multinomial resampling is applied for $t < T$.

After the final step, the unnormalized weights are stored as the final importance weights.

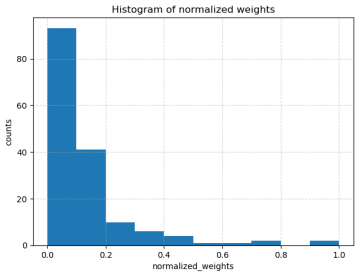
Observations from Table 4 [Task 3.]: TSMC has a similar setup to SMC but with a new twist function ϕ_t(looks one step ahead and adjusts the weights based on future rewards). Theoritically, these extra term would make stable updates leading to diverse yet within margin weights of particles[no change in entropy], thus having lower perplexity than Task 2. The only caveat with TSMC is that the reward gets reduced for better stability and diversity across the sampled.
Histograms Task 3: Unlike Task 2, the weight distributions stay relatively stable even for larger β values. ϕ_t smooths the update process, keeping particles more balanced and preventing early weight collapse.

Task 3	**Expected Reward**	**Perplexity**	**Entropy**
$\beta = 0.5$	7.783	5.77	5.376
$\beta = 1.0$	7.710	7.470	5.328
$\beta = 5$	6.804	7.575	5.093
$\beta = 10$	7.788	8.034	5.071

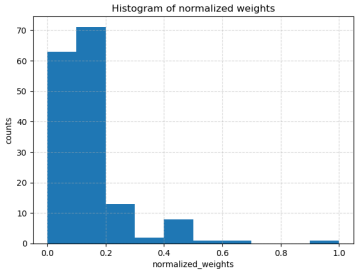
Table 4: Performance comparison on Task 3 for different β values.



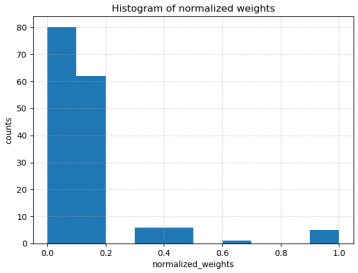
(a) $\beta = 0.5$



(b) $\beta = 1.0$



(c) $\beta = 5$



(d) $\beta = 10$

Figure 3: Histograms of Task 3 results for different β values.

5 Task 4: Free Reign on Sampling

In this work, we leverage the rich toolkit of Sequential Monte Carlo (SMC) for these probabilistic inference problems. In particular, we use learned twist functions to estimate the expected future value of the potential at each timestep, which enables us to focus inference-time computation on promising partial sequences. TSMC sequentially refines its sampling effort to focus exploration on promising candidates, resulting in more efficient generation of high-quality solutions. The main idea is to use twisting, an SMC technique that enjoys good computational efficiency, to incorporate heuristic approximations without compromising asymptotic exactness.

```
python3 task4.py --hf-token <<your_hf-token>> --device cuda:0 \
--counts-dir <dir_path> \
--A 2 --B 8 \
--out <jsonl_file_path> \
--k 10 --beta 5
```

Task 4	Expected Reward	Perplexity	Entropy
$k = 5$, $\beta = 0.5$	13.86	4.68	5.585
$k = 5$, $\beta = 1.0$	16.426	5.688	5.642
$k = 5$, $\beta = 5$	19.520	7.603	5.630
$k = 5$, $\beta = 10$	19.848	7.414	5.625
$k = 10$, $\beta = 0.5$	13.857	4.67	5.537
$k = 10$, $\beta = 1.0$	16.07	5.795	5.61
$k = 10$, $\beta = 5$	19.51	7.38	5.808
$k = 10$, $\beta = 10$	19.804	7.628	5.786

Table 5: Performance comparison on Task 4 with varying k and β values.

6 GenAI Usage

The specific part where we approximate bigram probabilities from the given trigram distribution was taken from a ChatGPT-generated snippet and later adapted to fit our setup[mentioned in the code as well].

References

[1] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The Curious Case of Neural Text Degeneration. 2020. In Proceedings of the International Conference on Learning Representations (ICLR). [https://arxiv.org/abs/1904.09751]

[2] Dan Jurafsky and James H. Martin. Speech and language processing (3rd ed. draft). 2023. Online draft. [https://web.stanford.edu/ jurafsky/slp3/]

[3] Yilun Hao, Haoyun He, Zihan Liu, Benjamin Van Durme, and Hao Luo. Probabilistic Inference in Language Models via Twisted Sequential Monte Carlo. 2023. In Proceedings of The Eleventh International Conference on Learning Representations (ICLR). [https://arxiv.org/abs/2305.15720]